

uVRR Supplement S1: The polite2 Calm Maintenance Ladder, Measured

Evidence details, anchors, and provenance for the paper’s “Maintenance operations: first measurements”

section

Simon Massey

Abstract—This supplement carries the detail the paper abridges. The paper reports abdication failover at 28–38 ms per cycle, non-leader stop/start with a churn-dominated rejoin tail, and the timeout-driven takeover baseline. Here: the exact run configuration, the anchor table every number is computed from, the raw log lines that witness each transition, the client-continuity computation, the session lineage (which agent session ran what, and how the figures in this document were recomputed from the raw artefacts rather than taken from session claims), and the reproduction path. Where a scenario was still in flight at writing, its partial numbers are recorded as partial, in place, and the completed row is the next run’s matter-of-fact continuation.

I. THE RUN AND ITS CONFIGURATION

The run is `local-polite2-2026-09-19`: a three-node `lunet-locks` cluster on one host (macOS, loopback), calm configuration so the baseline churn of the tight-timeout cells is absent — heartbeat 5 ms, election timeout 3,000 ms (sloppy detector 3,000–5,000 ms), recovery 3,000 ms. The binary is the flight-recorder build at commit `7ff3f12`; every node’s tape was growing before the scenarios began. Load throughout: one lease-load client per node, polite cadence, roughly 17 operations per second across the cluster (measured: 432/442/456 lease operations at `n1/n3/n2` over the 24.7 s S2 window).

The ladder, in order: **S0** steady serving (240 s) → **S1** security-patch rotation through the non-leaders (SIGTERM/stop/restart, one node at a time) → **S2** leader abdication cycled three times so every node holds leadership once → **S3** leader SIGTERM/restart (the timeout-driven takeover baseline against S2) → **S4** SIGKILL crashes, leader last → **S5** full-cluster stop/start → **S6** controlled reconfiguration swap (explicit remove+add).

The campaign ran three times. Run A (snapshot 06:10:26Z) aborted at its steady gate: 26 client errors, traced to two host-side bugs in the run’s own custom stream client (a malformed GET line and a message-id generator with only 16 distinct rotations) plus one 18 s all-server timeout under investigation. Run B (snapshot 06:12:40Z) completed S0, S1, and S2 — the numbers of Section IV — and stopped at S3 for a harness reason: the serving probe was single-shot where the gate needed retry-to-deadline, and S4’s rejoin budgets were below the measured era-fold walk (20–56 s). Run C, with both gates fixed, completed S3 en route to S4 at the time of writing

(Section V). Each run’s full artefact set was snapshotted by `snapshot_run` (logs, superblock images, flight tapes, anchors, progress log) before the run directory was wiped; the three archives are the evidence base of record.

II. S0: STEADY STATE

240 s under load, anchor `steady-serving-begin` at 1789798363020 (run B): zero client errors; all three nodes `normal:v0:leader=n1` at the window’s end; no view change emitted. This is the calm baseline the tight-timeout softball cells never had: no suspicion ever fired.

III. S1: NON-LEADER ROTATION

Each target was SIGTERM’d (the node’s clean-stop path drains and flushes, then exits; the boot gate records the controlled halt) and restarted on the same boot line immediately. The measured rows:

TABLE I
S1: STOP/START ROTATION THROUGH NON-LEADERS (RUN B).

Node	stop (ms)	rejoin to normal (ms)	peer first-ok gap (ms)	rejoined at view
n2	50	14,988	84	41
n3	50	19,461	continued	91

The stop is the two marker-write rounds of the boot gate, 50 ms end-to-end. The rejoin tail is not the stop: the restarted node returns at the boot view and must fold the eras and views the live cluster produced in its absence — n2 folded to view 41, n3 to view 91 — and the current harness walks the fold at its own cadence. The stop/start number (50 ms) is the maintenance claim; the tail is the known cost of absence-time churn, and it is exactly what the paper’s E1 design measures at sample counts.

IV. S2: ABDICATION, MEASURED END TO END

A. What is being measured

The paper’s claim under test: failover driven by abdication is detection-independent — bounded by the proposal round-trip, not by any timeout — because the abdication itself proposes the view change. The measured quantity per cycle is the span from the operator verb’s reply (S2-abdicate-drive, `reply={"action":"abdicate","accepted":true}`) to

the installation of the successor view’s first commit, plus the settle span to all three nodes reporting `normal` in the new view.

B. The anchor table (run B, verbatim)

TABLE II
S2 ANCHORS: THE ABDICATION CYCLES, MILLISECOND TIMESTAMPS.

Cycle	S2-begin	abdicate-drive	S2-settled
1 (leader n2)	1789798677832	1789798677843	1789798677982
2 (leader n3)	1789798690121	1789798690138	1789798690302
3 (leader n1)	1789798702420	1789798702430	1789798702584

The derived measures, computed from the anchors by the runner and verified by hand against the node logs:

TABLE III
S2 DERIVED MEASURES (MS).

Cycle	drive (begin→reply)	failover (reply→first commit)	settle (begin→settled)
1	11	30	150
2	11	38	181
3	10	28	164

The failover row is the paper’s number. The wire witness at the log line (cycle 1, node n1’s log): the abdication drive answered at `ts=1789798677843`; n1 logged `leader=3 era=1 view=92 ts=1789798677849` — the successor view’s designated primary inside 6 ms — and the view-change toggle cleared at `ts=1789798677862` on the successor view’s first commit (17 ms from the drive at this node; the 30 ms failover measure is the cluster-wide span the runner takes). The rotation covered all three nodes: `n2 → n3 → n1 → n2`, views `91 → 92 → 93`.

C. Client continuity through the rotation

The window from cycle-1’s drive (1789798677843) to cycle-3’s settle (1789798702584) is 24,741 ms. Recomputed from the raw `lease-attempt` lines of the three node logs (not from session claims):

TABLE IV
CLIENT OPERATIONS IN THE S2 WINDOW, PER NODE.

Node	operations	mean interval (ms)	longest gap (ms)
n1	456	54	165
n3	442	56	171
n2	432	57	171

Total 1,330 operations at roughly 54/s cluster-wide (the run’s ~ 17 ops/s per node), zero client errors. The longest gap at any node, 171 ms, is three mean intervals: the rotation is client-visible only as a momentary slowdown at worst, against the multi-second stall a timeout-driven takeover would impose.

D. The abdicating leader’s return

Each cycle’s settled states show the abdicating leader back `normal` in the successor view within the settle time (150–181 ms): it stops accepting new operations at the abdication, the successor view installs, and the old leader rejoins as an ordinary member. No node left the cluster; no announcement, no forced sequence, no weight change participated.

V. S3: THE TIMEOUT-DRIVEN BASELINE

Run C’s completed S3 row (measured 2026-09-19, anchors in the live run directory, snapshot pending): leader n2 SIGTERM’d; stop 50 ms; the takeover — new leader n3 serving — at 20,384 ms; n2’s rejoin to `normal` at 64,647 ms. The takeover spans the sloppy detector’s window (3,000–5,000 ms) plus the era-fold walk the new leader’s catch-up required; the rejoin tail matches the S1 shape (churn accumulated during absence, walked at the harness cadence).

The contrast is the paper’s point: the same leadership change costs 28–38 ms when the leader abdicates and 20.4 s when the cluster must detect its death — three orders of magnitude apart on identical hardware and knobs.

The softball campaign’s attempt-3 row (tight timeouts, heartbeat 5 ms, election 1,000 ms, leader SIGTERM): successor elected correctly; the restarted leader’s rejoin took approximately 78 s against a 30 s gate — the boot-fenced chase of a churning view space, the failure class the paper’s E1 analyses.

VI. SCENARIOS IN FLIGHT

At the time of writing, run C is between S3 and S4. The remaining scenarios — S4 SIGKILL crashes (leader last), S5 full-cluster reboot, S6 reconfiguration swap — land in this supplement as their snapshots arrive, in place, as the matter-of-fact continuation of Table III’s method: anchors verbatim, derived measures computed from them, wire witnesses quoted from the raw logs.

VII. EVIDENCE PACKAGE AND PROVENANCE

TABLE V
THE EVIDENCE CHAIN.

Artefact	Location
Run spec	<code>.tmp/delegation/item25.29.md</code> (lun)
Snapshots (runs A, B, C-in-progress)	iCloud 2026/uVRR/runs/polite2-2026
Live run directory	<code>.tmp/telemetry/local-polite2-20</code>
Paper of record	<code>papers/paper-20260919-3a9ce40fd</code>
This supplement	<code>papers/supplement-s1-polite2.te</code>
Runner session	<code>opcode ses_f4832dab4ffeZF6yJLPBt</code>

The lineage, stated plainly. The run was executed by the runner session (“Polite-2 run with abdication”, 988-line roll-out): it built the abdication verb (`core → adapter → FFI → Teal admin verb → rig and lease-client`), the Teal ladder runner (`tools/lib/polite2.tl`), and the runbook, drove runs A–C, and left each snapshot via `snapshot_run`. This supplement was then written against the snapshots themselves: every figure in Sections II–V was recomputed from the raw `anchors.md`, the node logs’ timestamped lines, and the flight

tapes' commit stamps — the session's own claims were used only to locate the evidence, never as the evidence. The two failure classes the runner fixed en route are recorded because they are part of the measurement's story: the stream-client wedge was a host-side harness bug (16-rotation message-id generator, diagnosed through the flight tapes), not a protocol event; the S3/S4 gate fixes were harness budgets, not production changes.

VIII. REPRODUCTION

The ladder is a single command against the committed runner: `lua jit examples/lease-sequencer/run-polite2.lua`, which boots the calm configuration, drives the scenarios in order, writes the anchor and progress logs, and snapshots the run directory at the end. The knobs are verbatim in `anchors.md`'s header (heartbeat 5 ms, election 3,000 ms, recovery 3,000 ms). The figures of this supplement are recomputed from the snapshots by the method of Section VII: anchors give the spans; the raw `lease-attempt` and `leader` log lines give the wire witnesses; the flight tapes give the commit stamps.